

Vectorization approach:

SEQUENCE 1
→ k-mer 분석
→ 벡터로 변환
→ 벡터 1536D

비교: 두 1536차원 벡터 거리 계산
시간 복잡도: $O(1536) = \sim$ 수천 번의 연산
결과: 100만 시퀀스 검색도 초 단위로 완료

DC:Dr 과h

Ce보본반상

~1→t갈 C

DNA 서열: ATCGATCGATCG

k=3 (3-mer 추출):
ATC, TCG, CGA, GAT, ATC, TCG, CGA, ATC, TCG, ATG...

k-mer 빈도 벡터:
전체 가능한 3-mer: $4^3 = 64$ 가지
각 3-mer의 빈도를 64차원 벡터로 표현

예시:
AAA: 0, AAC: 0, AAG: 1, AAT: 0,
ACA: 2, ACC: 0, ACG: 1, ACT: 0,
...
TTT: 0, TTC: 1, TTG: 0, TTT: 0

결과 벡터: [0, 0, 1, 0, 2, 0, 1, 0, ..., 0, 1, 0, 0]

C

--	--

과11111과11111과

	<m~,~F: 벡><0~F< 벡 6=>-
	,Y,가-
	,
	-

~ C

k=2: 16차원, 너무 간단 (기능 구분 어려움)
k=3: 64차원, 일반적 (빠르고 충분한 정보)
k=4: 256차원, 더 정확 (계산 비용 증가)
k=5+: 고차원, 느림 (실시간 검색 부적절)

De

NXJ6dtr 3NXJLO`bC

전통적 NLP 모델을 DNA에 적용:

- DNABERT (DNA BERT):
- DNA를 "토큰"으로 취급 (k-mer 또는 코돈)
 - 사전학습 언어 모델 (BERT)을 DNA 데이터로 미세조정
 - 각 DNA 서열을 고정 길이 벡터로 변환
 - 유사한 기능의 DNA는 벡터 공간에서 가까이 위치

장점:

- 생물학적 의미를 포착 (기능 유사성)
- 1536차원 밀집 벡터 (정보 손실 적음)
- 사전학습으로 일반화 성능 우수

단점:

- 계산 비용 높음 (GPU 필요)
- 블랙박스 모델 (해석 어려움)
- 재학습 비용

Me

DNABERT 임베딩 + k-mer 특징의 하이브리드:

- DNABERT: 의미 정보 (기능 유사성)
- k-mer: 해석 가능 정보 (서열 특성)

1. DNABERT로 1536D 벡터 생성
2. k-mer 특징 64D 추가
3. 총 1600D 벡터로 검색

이점:

- 검색 정확도 향상 (두 관점 모두 고려)
- 결과 해석 가능 (k-mer 기여도 확인)

DC:Mr 과

1. 쿼리 DNA 서열 입력
예: 새로운 박테리아 서열

↓

2. 벡터 변환
 - k-mer 추출
 - DNABERT 임베딩
 - 결과: 1536D 벡터

↓

3. Milvus에서 유사 검색
```  
SELECT id, species, sequence  
FROM dna\_collection  
WHERE embedding <-> query\_vec < threshold  
LIMIT 10  
```  
결과: 유사한 서열 10개

↓

4. 분류 결정
 - 검색된 서열의 종(species)을 확인
 - 투표 방식: 상위 10개 중 가장 많은 종
 - 신뢰도: 최상위 결과의 거리로 측정
 - 결과: 새로운 서열 = "종 X"로 분류

```
import milvus
import numpy as np
from sklearn.preprocessing import normalize

# 1. Milvus 연결
client = milvus.Milvus(host='localhost', port=19530)

# 2. DNA 벡터화
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("dnabert")
model = AutoModel.from_pretrained("dnabert")
```

```
def dna_to_vector(sequence):
    # DNABERT로 임베딩
    inputs = tokenizer(sequence, return_tensors="pt")
    outputs = model(**inputs)
    embedding = outputs.last_hidden_state[:, 0, :].detach().numpy()
    return normalize(embedding)[0] # 정규화

# 3. 쿼리 실행
query_sequence = "ATCGATCGATCGATCGATCGATCG..."
query_vector = dna_to_vector(query_sequence)

# 4. Milvus에서 검색
results = client.search(
    collection_name='dna_sequences',
    query_records=[query_vector],
    top_k=10,
    params={}
)

# 5. 결과 분석
species_votes = {}
for result in results[0]:
    species = metadata[result.id]['species']
    species_votes[species] = species_votes.get(species, 0) + 1

predicted_species = max(species_votes, key=species_votes.get)
confidence = max(species_votes.values()) / 10 # 상위 10개 중 비율
```

DC:N

e

0

C

52

O

e RO`0개한ryt개F-02r각yt

JXN vt 기림개 + 0개xt 잘 xyp+

JXN ~ 각 $\rightarrow$ F $\rightarrow$ pr 개 잡 +

0

62

0

e RO`0개 검가 tnt가 저 LObe 00X 54444 JXN =4444

J XN v r nr 각 개 가 개 L obe 00X 42= J XN 42=

J XN xp개각관 가결psy가vnu결→t F 재결

0

$$: 2$$

O

e RO`0개 ~~원~~ nsp가 pqp개 F XMLS

JXN 개 값 가가 G 6464145145+

JXN 개입검가 $\gamma_{kn \rightarrow t}$ 저각 F-검 γ_{kt}

0

<2

0

$$e \in R \implies 0 \leq \text{rank}(e) \leq \text{rank}(1) = n$$

JXN 각말 p가개 $\rightarrow$ n개 p가개 + 간b저각t가r+

JXN p가타 n가타 t & XYb Xc W

0

```
-- 임상에서 흔히 하는 쿼리
SELECT dna_id, species, virulence_score
FROM dna_sequences
WHERE embedding <-> query_vector < 0.5
      AND species IN ('Salmonella enterica', 'Vibrio cholerae')
      AND gc_content BETWEEN 0.4 AND 0.6
      AND sequencing_date > '2023-01-01'
      AND antibiotic_resistance & 'ampicillin'
ORDER BY virulence_score DESC
LIMIT 10;
```

C

Ct→qtssy가 E1G 감염 감지 r 거발 42
 C가한 ry 개SX, 222
 Cvrnr 각 매가거 Obe 00X 42 J XN 42
 C개 감염 가가 nsp 게 G 646: 145145+
 Cp가거 n가 n가 n가 n가 t * p→간 ry가

dJ_,dtr 거발 검→t 거s _ 감 감형 2

DC:S 계비생

Wy것개

C

```
# Milvus 스키마 정의
from milvus import FieldSchema, CollectionSchema

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536),
    FieldSchema(name="species", dtype=DataType.VARCHAR),
    FieldSchema(name="gc_content", dtype=DataType.FLOAT),
    FieldSchema(name="sequence_length", dtype=DataType.INT64),
    FieldSchema(name="sequencing_date", dtype=DataType.VARCHAR),
]

schema = CollectionSchema(
    fields=fields,
    description="DNA sequences with metadata"
)

client.create_collection(
    collection_name='dna_sequences',
    schema=schema
)

# 벡터 + 메타데이터 함께 삽입
entities = [
    [id_1, id_2, ...], # id
    [vec_1, vec_2, ...], # embedding
    ['E. coli', 'Bacillus', ...], # species
    [0.48, 0.52, ...], # gc_content
    [5000000, 3000000, ...], # sequence_length
    ['2023-01-15', '2023-02-20', ...], # sequencing_date
]

client.insert(
    collection_name='dna_sequences',
    records=entities
)

# 메타데이터 필터와 함께 벡터 검색
expr = "species == 'E. coli' AND gc_content > 0.4"
results = client.search(
    collection_name='dna_sequences',
    query_records=[query_vector],
    top_k=10,
```

```
expr=expr # 메타데이터 필터 조건
)
```

Wy것개 C
/ 0
0 NL

DC:[
C
2 ? 3

웹 검색, 추천 시스템, 이미지 검색
→ 사용자가 정의한 "유사성" 개념이 분명함
→ 벡터 검색이 주요 기능

2과h6 6 3
서열 분류, 구조 검색, 물성 예측
→ 도메인 전문 지식 + 벡터 검색이 결합됨
→ 필터링, 조인, 집계가 필수
→ VAQ가 필수

C
인간 게놈:
- 크기: ~3십억 뉴클레오티드
- 유사도 검색 시간: 순차 검색으로 분 단위
- 벡터 검색으로: 밀리초 단위 가능
단백질 서열 데이터베이스 (UniProt):
- ~2억 개의 단백질 서열
- k-mer 필터링 + 벡터 검색 → 실시간 가능
미생물 메타게놈:
- 환경 샘플에서 수백만 개의 서열 추출
- 종 분류, 기능 주석을 동시에 수행

DC:]
dJ_ C

dJ_ C
52
o개
11
aOVOMb . P`YWsgpnsq
e RO`Ot→qtssyg E1G 검 검r E 42
11
aOVOMb . P`YWsgpnsq
e RO`Ot→qtssyg E1G 검 검r E 42
JXN 개anryt개 거갈t개개anryt개
JXN vnrnr각머가G 42
JXN 개검가가vn검by경G=4
Y`NO` Lg 개→yp검NOaM
0
62 /
Wy것개
^각머갈a_V/ 간것r거말
0검검거말 /
: 2

o가
aOVOMb가
P`YWsg
TYX r
e RO`O
JXNr2
Q`Yc^ Lg
Y`NO` Lg r
o

C

52
NXJLO`b
3
62
XMLS
Wy
: 2
a_V aOVOMb
2

C:

NXJ
C
C
C

0& & 2

D:

C

- 1. 벡터 검색으로 후보 1000개 추출
 - 2. 그 중 메타데이터 필터 적용
- 문제: 필터 선택도 낮으면 유효 결과 부족

전략 2: 필터 먼저

- 1. 메타데이터 필터로 데이터 축소
- 2. 축소된 데이터에서 벡터 검색

문제: 필터 선택도 낮으면 인덱스 활용 안 됨

전략 3: 동시 처리 (NHQ의 아이디어)

필터와 벡터를 그래프에서 동시에 고려

문제: Milvus는 미지원, SQL 기반 DB 필요

M

C

XML, Sqt가, p가, C

C

52, Q^c -

62Wy것개

: 2

2